



CS319

First Deliverable

Team 6

Section 1

Bora Yetkin - 22203661

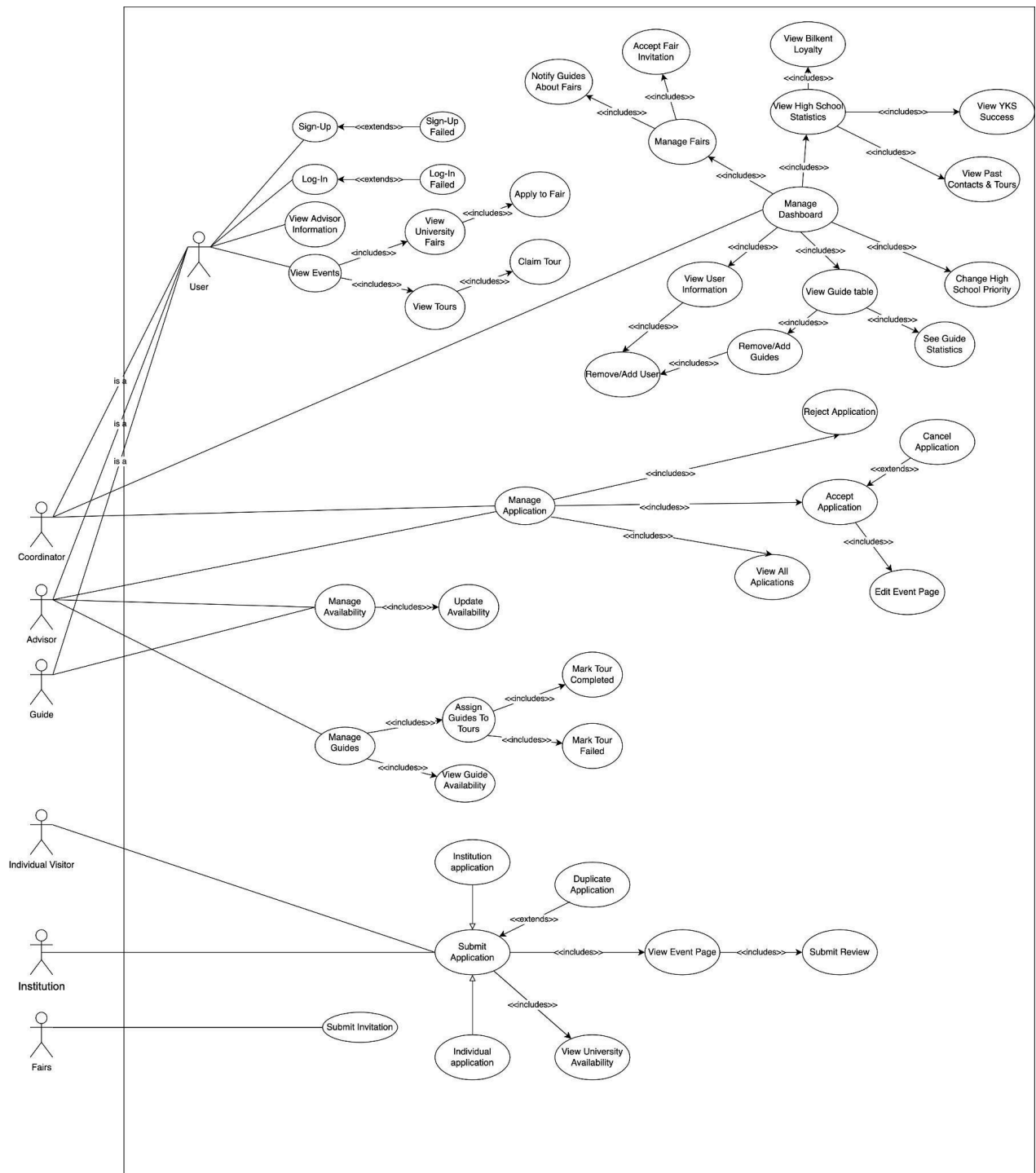
Erdem Pülat - 22103566

Halil Avcı - 22003476

Ramiz Arda Ünal - 22202554

Tuna Göksal - 22203827

Use Case Diagram



Use Case Descriptions

Use Case Name: **Sign Up**

Participating Actors: User

Entry Condition: When a user is not signed up and wants to create an account.

Exit Condition: The user is successfully signed up and redirected to the login page, or the process fails and they are notified.

The Flow of Events:

1. The user clicks the "Sign Up" button on the website.
2. The system displays the sign-up form.
3. The user fills out the required fields: name, email, password, and role (e.g., guide, advisor).
4. The system validates the inputs (e.g., valid email format, password requirements).
5. If validation passes, the system saves the user's details to the database.
6. The user is notified of successful sign-up and is redirected to the login page.
7. Use case ends when the user logs in or navigates away.

Use Case Name: **Sign-Up Failed**

Participating Actors: User

Entry Condition: When a user tries to sign up with invalid or incomplete information.

Exit Condition: The user is informed of the error and prompted to correct the information.

The Flow of Events:

1. The user attempts to sign up with invalid details (e.g., an invalid email format or missing required fields).
2. The system validates the inputs and detects an issue.
3. The system displays an error message to the user (e.g., "Email is invalid" or "All fields are required").
4. The user is prompted to correct the information and resubmit the form.
5. Use case ends when the user corrects the information or exits the sign-up page.

Use Case Name: **Log In**

Participating Actors: User

Entry Condition: When a user has an account and is not logged in.

Exit Condition: The user is successfully logged in and redirected to their dashboard or the homepage.

The Flow of Events:

1. The user navigates to the login page and enters their email and password.
2. The system checks if the email and password match an account in the database.
3. If the credentials are valid, the system generates a JWT (JSON Web Token) for the session.
4. The user is logged in, and the system redirects them to their dashboard or the homepage.
5. Use case ends when the user is successfully logged in.

Use Case Name: **Log-In Failed**

Participating Actors: User, Database

Entry Condition: When a user provides invalid login credentials (e.g., incorrect email or password).

Exit Condition: The user is notified of the invalid credentials and remains on the login page.

The Flow of Events:

1. The user enters their email and password and clicks the "Login" button.
2. The system checks the credentials against the database and finds them invalid (e.g., incorrect password or non-existent email).
3. The system displays an error message to the user (e.g., "Invalid credentials").
4. The user is prompted to correct their information and attempt login again.
5. Use case ends when the user corrects the credentials or exits the login page.

Use Case Name: **View Events**

Participating Actors: User

Entry Condition: The user selects the "Events" page

Exit Condition:

The Flow of Events:

1. The user clicks on the "Events" page icon
2. The user can select which type of events to view. The default option is the Tours.
3. The system shows a list of at most 30 closest events.
4. The user can click on any event to go to its event page, which exits the use case.

Use Case Name: **View University Fairs**

Participating Actors: User

Entry Condition: The user selects the "University Fairs" option in the "Events" page.

Exit Condition: The user clicks the button to exit the "University Fairs" page.

The Flow of Events:

1. The user clicks on the "University Fairs" option on the "Events" page
2. The system lists fairs, including assigned guides and advisors, the high school's priority, and the date, time, and place.
3. The user can click on any event to go to its event page, which exits the use case.

Use Case Name: **Apply To Fair**

Participating Actors: User

Entry Condition: The user selects the "University Fairs" option on the "Events" page, and there is still a need for guides for the tour

Exit Condition: The user successfully claims the tour, or the claim is unsuccessful

The Flow of Events:

1. The user clicks on "Apply to Fair" on the "University Fairs" Page on the "Events" page
2. The user is asked to confirm a second time with a confirmation prompt
3. The application is completed if the user confirms.
4. The user is then redirected to the "Events" page with the "University Fairs" filter.

Special / quality requirements: Confirmation prompt

Use Case Name: View Tours

Participating Actors: User

Entry Condition: The user selects the "Tours" option in the "Events" page

Exit Condition: The user clicks the button to exit the "Tours" page.

The Flow of Events:

1. The user clicks on the "Tours" option on the "Events" page
2. The system lists high school tours, including assigned guides and advisors, the high school's priority, and the date and time.
3. The users can claim tours that they want to attend.
4. The user can click on any event to go to its event page, which exits the use case.

Use Case Name: Claim Tour

Participating Actors: User

Entry Condition: The user selects the "Tours" option on the "Events" page, and there is still a need for guides for the tour

Exit Condition: The user successfully claims the tour, or the claim is unsuccessful.

The Flow of Events:

1. The user clicks on "Claim Tour" on the "Tours" Page on the "Events" page
2. The user is asked to confirm a second time with a confirmation prompt
3. The claim is completed if the user confirms.
4. The user is then redirected to the "Events" page with the "Tours" filter.

Special / quality requirements: Confirmation prompt

Use Case Name: View Advisor Information

Participating Actors: User

Entry Condition: When a guide wants to view contact information and details about the advisor for a tour.

Exit Condition: The guide successfully views the advisor's information or exits the page.

The Flow of Events:

1. The guide logs into their account and navigates to the "Advisor Information" page.
2. The system retrieves the advisors' details from the database, including their name, contact information, and assigned day.
3. The advisors' information is displayed to the guide in a list.
4. Use case ends when the guide closes the advisor information page or navigates to another page.

Use Case Name: Manage Dashboard

Participating Actors: Coordinator

Entry Condition: Coordinator logs in to the system and wants to access the dashboard.

Exit Condition: Coordinator logs out from the dashboard.

The Flow of Events:

1. The coordinator navigates to the dashboard.

2. The system displays an overview of statistics and guide performance along with an applications menu.
3. Coordinator can view the given information and manage applications.
4. Use case ends when the coordinator exits the dashboard.

Special / quality requirements:

Use Case Name: **Change High School Priority**

Participating Actors: Coordinator

Entry Condition: Coordinator accesses the dashboard and wants to make changes to High Schools with priority, chooses that option from the menu.

Exit Condition: The high school priority order is updated and saved in the system, confirmation is provided to the Coordinator.

The Flow of Events:

1. Coordinator navigates to the priority section of the dashboard.
2. Coordinator selects the high schools that need a change in priority.
3. Coordinator updates new priority order.
4. Coordinator saves changes, a message is displayed to confirm success.
5. System updates the priority list, applications are now viewed based on the updated priority list for advisors.
6. Special / quality requirements: Confirmation Prompt.

Use Case Name: **Manage Fairs**

Participating Actors: Coordinator

Entry Condition: User selects Manage Fairs option in the dashboard menu.

Exit Condition: User successfully viewed fair applications.

The Flow of Events:

1. User selects the "Manage Fairs." option.
2. System displays current fair invitations. System also displays the organization's name, phone number along with the name of the guides who applied to a given fair.
3. Coordinator can access other use cases related to managing fairs from this page

Special / quality requirements: Fair details should be accurate.

Use Case Name: **Accept Fair Invitation**

Participating Actors: Coordinator

Entry Condition: The user selects the accept or deny option in the Manage Fair page.

Exit Condition: The system has successfully updated the fair information and the coordinator managed invitations.

The Flow of Events:

1. The user selects a confirmation option.
2. The system confirms this option then updates the list in the Manage Fairs page as accepted or denied.

Special / quality requirements: Confirmation Prompt.

Use Case Name: **Notify Guides About Fairs**

Participating Actors: Coordinator

Entry Condition: The user selects the option to notify the guides in the Manage Fair page.

Exit Condition: Coordinator has successfully notified the guides.

The Flow of Events:

1. Coordinator selects the “Manage Fairs.” option.
2. System displays current fair invitations. System also displays the organization's name, phone number along with the name of the guides who applied to a given fair.
3. Coordinator confirms this information and selects the “Notify Guides” option for a given fair.
4. Guides who applied to the given event are informed via email.

Special / quality requirements: An option to notify all guides for all accepted fairs can be implemented.

Use Case Name: **View User Information**

Participating Actors: Coordinator

Entry Condition: The user selects the “View User Information” option from the dashboard menu.

Exit Condition: The coordinator has successfully viewed the requested information.

The Flow of Events:

1. The user selects “View User Information.”
2. The system displays details about users, including their roles, names and email addresses.
3. Special / quality requirements: Information must be updated after the coordinator makes a change.

Use Case Name: **Remove/Add User**

Participating Actors: Coordinator

Entry Condition: The coordinator selects the “Remove/Add User” option from the dashboard menu or accesses the functionality through other pages.

Exit Condition: The Coordinator has successfully added or removed a user.

The Flow of Events:

1. The Coordinator selects “Remove/Add User.”
2. The system presents a form to add a user or a list to remove a user.
3. The Coordinator inputs the necessary information and confirms the action.

Special / quality requirements: Confirmation Prompt. Should only take “Guide” as a user role if chosen from the “View Guide Table” page.

Use Case Name: **View Guide Table**

Participating Actors: Coordinator

Entry Condition: Coordinator selects the “View Guide Table” option from the dashboard menu.

Exit Condition: The user has successfully viewed the information given.

The Flow of Events:

1. The user selects “View Guide Table”
2. System presents a list of all active guides along with their information.
3. The user can view a given guide’s statistics or can remove or add a guide.

Special / quality requirements: Information must be updated after the coordinator makes a change.

Use Case Name: **See Guide Statistics**

Participating Actors: Coordinator

Entry Condition: Coordinator clicks on a guide from the guide table to see more about them.

Exit Condition: The user has successfully viewed the information given.

The Flow of Events:

1. The user selects a guide from the "Guide Table"
2. The system displays relevant statistics, such as the number of tours led by the given guide.

Special / quality requirements: Information must be updated when guides perform a new task.

Use Case Name: **Remove/Add Guides**

Participating Actors: Coordinator

Entry Condition: The coordinator selects the "Remove/Add Guide" option from the Guide Table page.

Exit Condition: The Coordinator has successfully added or removed a guide.

The Flow of Events:

1. The Coordinator selects "Remove/Add Guides."
2. The system presents a form to add a new guide or a list of guides to remove one.
3. The Coordinator inputs the necessary information and confirms the action.

Special / quality requirements: Confirmation Prompt. Should only take "Guide" as a user role.

Use Case Name: **View High School Statistics**

Participating Actors: Coordinator

Entry Condition: When a coordinator wants to analyze data related to high school visits.

Exit Condition: The coordinator navigates away from the statistics section or logs out.

The Flow of Events:

1. The coordinator logs in and navigates to the statistics section.
2. The coordinator selects the "View High School Statistics" option.
3. The system retrieves data from the database
4. The coordinator views detailed statistics on the selected high schools.

Use Case Name: **View Bilkent Loyalty**

Participating Actors: Coordinator

Entry Condition: When a coordinator is viewing high school statistics and wants to see the loyalty of high schools toward Bilkent University.

Exit Condition: The coordinator returns to the main statistics page, selects another option, or logs out.

The Flow of Events:

1. The coordinator selects the "View Bilkent Loyalty" option within the high school statistics.
2. The system retrieves data showing which high schools frequently visit Bilkent.
3. The system displays a loyalty score or ranking for each high school based on their visit frequency.

Use Case Name: **View YKS Success**

Participating Actors: Coordinator

Entry Condition: When a coordinator is viewing high school statistics and wants to see how high schools perform in YKS

Exit Condition: The coordinator navigates back to the previous page, selects another option, or logs out.

The Flow of Events:

1. The coordinator selects the "View YKS Success" option within the high school statistics.
2. The system retrieves data related to the success rates of students from different high schools in YKS.
3. The system displays this data in a tabular or graphical format.

Use Case Name: **View Past Contacts & Tours**

Participating Actors: Coordinator

Entry Condition: When a coordinator is viewing high school statistics and wants to review historical data of high school visits and contacts.

Exit Condition: The coordinator navigates back to the statistics overview page, selects another option, or logs out.

The Flow of Events:

1. The coordinator selects the "View Past Contacts & Tours" option within the high school statistics.
2. The system retrieves past records of tours conducted and contacts made with each high school.
3. The system displays the data, showing dates, number of students, and outcomes of each visit.

Use Case Name: **Submit application**

Participating Actors: Institution, individual visitor

Entry Condition: Institution or Individual Visitor selects "submit application" option

Exit Condition: The application has been successfully submitted, or an error message is displayed if the submission failed.

The Flow of Events:

1. The actor navigates to the "Submit Application" page.
2. The system prompts the actor to provide necessary details, such as name, contact information, institution name (if applicable), number of visitors, and the preferred visit date.
3. The actor fills in the required information and submits the form.
4. The system validates the input and checks for errors or missing data.
5. If the data is valid, the system sends verification email to the actor and the actor should process it with verification code.
6. If the verification code is valid, the system stores the application and confirms the submission by showing a success message.
7. If there are validation errors (e.g., missing fields or incorrect data), the system displays an error message and prompts the actor to correct the information.

Use Case Name: **Institution application**

Participating Actors: Institution

Entry Condition: Institution enters submit application page

Exit Condition: The application has been successfully submitted, or an error message is displayed if the submission failed.

The Flow of Events:

1. The Institution representative navigates to the "Submit Application" page.
2. The system prompts the Institution representative to provide necessary details, such as name, contact information, institution name , number of visitors, and the preferred visit date.
3. The Institution representative fills in the required information and submits the form.
4. The system validates the input and checks for errors or missing data.
5. If the data is valid, the system stores the application and confirms the submission by showing a success message.
6. If there are validation errors (e.g., missing fields or incorrect data), the system displays an error message and prompts the actor to correct the information.

Use Case Name: **Individual Application**

Participating Actors: Individual Visitor

Entry Condition: Individual visitor enters the submit application page

Exit Condition: The application has been successfully submitted, or an error message is displayed if the submission failed.

The Flow of Events:

1. The individual visitor navigates to the "Submit Application" page.
2. The system prompts the individual visitor to provide necessary details, such as name, contact information, student's major of interest, number of visitors, and the preferred visit date.
3. The actor fills in the required information and submits the form.
4. The system validates the input and checks for errors or missing data.
5. If the data is valid, the system stores the application and confirms the submission by showing a success message.
6. If there are validation errors (e.g., missing fields or incorrect data), the system displays an error message and prompts the actor to correct the information.

Use Case Name: **Duplicate Application**

Participating Actors: Institution, Individual Visitor

Entry Condition: Institution or individual visitor that already has an application tried to make another one.

Exit Condition: Actor viewed the message that an application already made in their name

The Flow of Events:

1. The actor attempts to submit a new application while an existing one is already on record.
2. The system checks the actor's information and detects an existing application linked to their identity.
3. The system blocks the submission and displays a message indicating that a previous application has already been submitted.

Use Case Name: **View University Availability**

Participating Actors: Institution, Individual Visitor

Entry Condition: Actor selects "View university availability" on the submit application page

Exit Condition: The actor has successfully viewed the information given.

The Flow of Events:

1. The actor clicks on the "View University Availability" option on the submit application page.
2. The system fetches and displays a list of available dates and times for visits.
3. The actor reviews the availability information, including any relevant details such as open dates, time slots and busy days.
4. The actor exits the availability view or proceeds to select a date for their application.

Use Case Name: **View Event Page**

Participating Actors: Institution, Individual Visitor

Entry Condition: Actor selects "view event page" after their application is approved

Exit Condition: The actor has successfully viewed the information given.

The Flow of Events:

1. After receiving approval for their application, the actor selects the "View Event Page" option.
2. The system retrieves event-related information, such as the visit schedule, assigned guide, meeting point, and any additional instructions.
3. The actor reviews the event details and prepares for the visit.
4. The actor exits the event page after viewing the necessary information.

Use Case Name: **Submit Review**

Participating Actors: Institution, Individual Visitor

Entry Condition: Actor selects "submit review" after they finished their tours.

Exit Condition: Actor made a review on their guide

The Flow of Events:

1. After completing their university tour, the actor navigates to the "Submit Review" section.
2. The system prompts the actor to provide feedback, including a rating and comments on the tour guide's performance.
3. The actor fills in the review form and submits their feedback.
4. The system processes and stores the review.
5. A confirmation message is displayed once the review is successfully submitted.

Use Case Name: **Manage Guides**

Participating Actors: Advisor

Entry Condition: Advisor selects "manage guides" page

Exit Condition: The advisor successfully views and manages guide and tour operations or exists page

The Flow of Events:

1. The advisor selects the "Manage Guides" option from the main menu.
2. The advisor can assign or reassign guides to different tours.
3. The system confirms any changes made and updates the guide database.
4. The advisor exits the page or continues managing guide operations.

Use Case Name: **View Guide Availability**

Participating Actors: Advisor

Entry Condition: Advisor selects the "View Guide Availability" option.

Exit Condition: The actor has successfully viewed the information given.

The Flow of Events:

1. The advisor selects the "View Guide Availability" option.
2. The system displays a list of all guides with their availability schedules.
3. The advisor reviews the availability data, identifying which guides are available for upcoming tours.
4. The system provides filtering options (e.g., by date or specific guide).
5. The advisor exits the page or continues with other tasks.

Use Case Name: **Assign Guides to Tours**

Participating Actors: Advisor

Entry Condition: Advisor selects the "Assign Guides to Tours" option.

Exit Condition:

The Flow of Events:

1. The advisor selects the "Assign Guides to Tours" option.
2. The system displays a list of upcoming tours that require guide assignments.
3. The advisor selects a tour and chooses from a list of available guides based on their availability.
4. The system assigns the selected guide to the tour.
5. The advisor confirms the assignment, and the system updates the tour and guide information.
6. The advisor exits the page or continues assigning more guides.

Use Case Name: **Mark Tour Completed**

Participating Actors: Advisor

Entry Condition: Advisor checks the checkbox with label "tour completed "

Exit Condition: The selected tour is marked as completed in the system.

The Flow of Events:

1. The advisor selects a tour from the "Manage Tours" section.
2. The system displays details of the tour, including the assigned guide.
3. The advisor checks the checkbox labeled "Tour Completed."
4. The system updates the tour status to "Completed" and notifies the relevant parties (e.g., guide, institution).
5. The advisor exits the page or continues managing other tours.

Use Case Name: **Mark Tour Failed**

Participating Actors: Advisor

Entry Condition: advisor empties the checkbox with label "tour completed "

Exit Condition: The selected tour is marked as failed in the system.

The Flow of Events:

1. The advisor selects a tour from the "Manage Tours" section.
2. The system displays details of the tour, including the assigned guide.
3. The advisor empties the checkbox labeled "Tour Completed."
4. The system updates the tour status to "Failed" and notifies the relevant parties (e.g., guide, institution).

5. The advisor exits the page or continues managing other tours.

Use Case Name: **Submit Invitation**

Participating Actors: Fair organizers

Entry Condition: Fair organizers select "submit invitation" option

Exit Condition: The invitation has been successfully submitted, or an error message is displayed if the submission failed.

The Flow of Events:

1. The fair organizers select the "Submit Invitation" option from the system menu.
2. The system prompts the organizers to enter invitation details, such as event date, venue, and target audience.
3. The organizers fill in all required fields and can review or edit the information before submission.
4. The organizers submit the invitation.
5. The system validates the provided information.
6. If all information is correct, the system successfully records the invitation and sends a confirmation message to the organizers.
7. If any required fields are missing or incorrect, the system displays an error message, and the organizers are prompted to correct the issues.
8. The fair organizers exit the page.

Use Case Name: **Manage Availability**

Participating Actors: Advisor, Guide

Entry Condition: Advisor/Guide selects the "Availability" page.

Exit Condition: Advisor/Guide clicks the button to exit the "Availability" page.

The Flow of Events:

1. The Advisor/Guide clicks on the "Availability" page icon
2. The Advisor/Guide can view and update their weekly availability.

Use Case Name: **Update Availability**

Participating Actors: Advisor, Guide

Entry Condition: Advisor/Guide selects the "Availability" page and then clicks on "Edit"

Exit Condition:

The Flow of Events:

1. The Advisor/Guide clicks on "Edit" while on the "Availability" page
2. The Advisor/Guide selects the hours they are available in a week.
3. The Advisor/Guide clicks on "Save"
4. The Advisor/Guide is asked to confirm a second time with a confirmation prompt
5. The Advisor/Guide is redirected to the "Availability" page.

Special / quality requirements: Confirmation prompt

Use Case Name: **Manage Application**

Participating Actors: Advisor, Coordinator

Entry Condition: User selects the "Manage Application" option from the dashboard.

Exit Condition: The user has successfully managed the application (e.g., edited, accepted, or rejected it).

The Flow of Events:

1. The user selects the "Manage Application" option.
2. The system displays a list of submitted applications with options to edit, accept, or reject.
3. The user chooses an action (e.g., accept, reject, edit details).
4. The system processes the selected action and updates the application accordingly.
5. A confirmation message is displayed.
6. Use case ends when the coordinator exits the page.

Special / quality requirements: Applicants and users are informed about the changes that affect them.

Use Case Name: **View All Applications**

Participating Actors: Advisor, Coordinator

Entry Condition: User selects the "View All Applications" option from the dashboard.

Exit Condition: User has successfully viewed all the applications.

The Flow of Events:

1. The user selects the "View All Applications" option.
2. The system retrieves and displays a list of all submitted applications.
3. The user can filter or sort the list based on various criteria (e.g., date, status).
4. Use case ends when the user exits the page.

Special / quality requirements: Information should be displayed in a paginated format to handle large numbers of applications.

Use Case Name: **Reject Application**

Participating Actors: Advisor, Coordinator

Entry Condition: User accesses the application management page and selects an application to reject.

Exit Condition: The application status is updated to "Rejected" in the system.

The Flow of Events:

1. The user navigates to the application management page.
2. The user selects an application and chooses the "Reject" option.
3. The system updates the status of the selected application to "Rejected."
4. A confirmation message is displayed to the user.
5. Rejection mail is sent to the applicant.

6. Use case ends when the user exits the application management page.

Use Case Name: **Accept Application**

Participating Actors: Advisor, Coordinator

Entry Condition: User accesses the application management page and selects an application to accept.

Exit Condition: The application status is updated to "Accepted" in the system.

The Flow of Events:

1. The user navigates to the application management page.
2. The user selects an application and chooses the "Accept" option.
3. The system updates the status of the selected application to "Accepted."
4. A confirmation message is displayed to the user.
5. Acceptance mail is sent to the applicant.
6. Use case ends when the user exits the application management page.

Use Case Name: **Cancel Application**

Participating Actors: Advisor, Coordinator

Entry Condition: User selects the "Cancel Application" option from the application page.

Exit Condition: The application is marked as "Canceled" in the system.

The Flow of Events:

1. The user navigates to their submitted application.
2. The user selects the "Cancel Application" option.
3. The system asks for confirmation before proceeding with the cancellation.
4. If the user confirms, the system updates the application status to "Canceled."
5. A confirmation message is displayed to the user.
6. Use case ends when the user exits the application page.
Special / quality requirements: Unlike reject application, canceled applications cannot be rearranged.

Use Case Name: **Edit Event Page**

Participating Actors: Advisor, Coordinator

Entry Condition: User selects the "Edit Event Page" option from the events management menu.

Exit Condition: The event page is successfully updated with the changes.

The Flow of Events:

1. The user navigates to the event management menu.
2. The user selects an event and chooses the "Edit Event Page" option.
3. The system displays the editable fields for the selected event (e.g., date, time, description).
4. The user makes changes and submits the form.
5. The system validates the input and updates the event details.
6. A success message is displayed.
7. An informative mail is sent to the correspondents.

8. Use case ends when the user exits the page.

Special / quality requirements: Input validation to ensure correct formatting of dates and other fields.

TECH STACK

To implement our Bilkent Tanitim Ofisi project we have chosen the MERN technology stack. Stands for MongoDB, Express.js, React.js and Node.js Used for full-stack development and known for efficiency and scalability.

Database: MongoDB

MongoDB is a database that stores data in documents similar to JSON. It's designed for scalability and performance. Especially this makes it ideal for our application. We chose MongoDB for our database because of its schema-less nature, which allows for easy modifications as our application evolves. MongoDB especially comes handy in scalability. This system is built to serve many guides and advisors, as well as using the database for 55000 high school students on tours. As the system grows MongoDB can scale as well. Data is always available for adding/removing operations. System can also handle multiple users simultaneously.

Data is especially important for this project as stated in the slides. MongoDB allows us to add the list of all high schools to the project for application processes. Wanted/good high schools for prioritization can be implemented easily, and changed when the coordinator wants. The scalability of Mongo allows for all new data to change the system all together. For preserving and visualizing the data in the Dashboard Atlas Charts can be used. With this tool data can be processed with a variety of metrics, calculations can be made on the data.

Backend: Node.js

Node.js is an environment used for executing the javascript on the server side. We chose Node.js because it works well with the rest of the system. Node.js is really good in real time applications, allowing our system to manage multiple users simultaneously. Implementation of adding and removing users, taking fair applications and assigning people can be coded with Node and Express so that it works well with the backend logic. This provides the flexibility to handle user interactions with the backend.

Backend: Express.js

Express.js is a minimal and flexible Node.js web application framework for building web and mobile applications. It is used with Node to construct the webpage and use API's. We choose Express.js as our backend framework because of its integration with Node.js. With Express we can use middleware to handle authentication which is useful for all kinds of user logins to our system like coordinator, guides and advisors. Using API endpoints is easy with Express. Assigning guides, adding users, taking tour implementations can be coded with Node and Express so that it works with the backend logic. This enables us to create scalable APIs. JWT is commonly used in Express.js for stateless authentication, which means you don't need to store session data on the server. Instead, JWTs are signed tokens that are sent along with each request, usually in the Authorization header.

Frontend: React

React means we can reuse the UI components easily, which is consistent for all of our designs. Different pages that display guides or show all tours can be shown with the same style and implementing this would be easier. Advisors can interact with dynamic data like an

advisor's schedule change can affect the page for guides with proper state updates. Using axios mongoose in React can easily allow us to build a form that submits all data to the backend which is a function that we will use a lot in this project. React's hooks are going to be used for tour applications and removing functions so managing user inputs is simplified.